Day18 模型前向推理测试（手写小GPT计划）

一、Day18 核心目标（贴合周3计划）

本周核心任务是“堆叠Transformer Block，完成GPT结构”，Day18作为Day17的延伸的关键测试环节，需实现：

1. 复用Day17定义的完整MiniGPT模型，完成全流程前向推理测试（覆盖CPU/MPS/GPU三种设备）
2. 验证模型输入输出的合理性，排查前向推理中常见报错（维度不匹配、设备不一致等）
3. 结合周1Day6的GPT专用数据格式，测试“真实输入序列”的推理效果
4. 为Day19参数量统计、Day21 `model.generate()` 雏形铺垫，确保模型可正常复用

二、前期组件复用（Day17核心内容，直接复用）

Day18无需重新定义模型，复用Day17完整MiniGPT模型及所有依赖组件，重点聚焦“测试”环节，确保组件兼容性：

- 完整MiniGPT模型（含Token Embedding、位置编码、Transformer Block堆叠、输出头）
- 所有基础组件：Mask机制、Scaled Dot-Product Attention、Multi-Head Attention、FFN等
- 设备配置逻辑（适配CUDA/MPS/CPU，周1Day3内容）

注：复用代码保留核心结构，优化测试相关细节，确保测试流程流畅可复现。

三、Day18 核心：模型前向推理测试（完整可运行）

重点实现“多设备测试+真实序列测试+报错排查”，贴合周3计划要求，为后续生成任务筑牢基础，代码可直接运行、可复现。

代码块

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import os

# 适配Mac MPS/GPU/CPU，贴合周1Day3 CUDA基础（保留Day17配置）
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"

# ------------------------------------------------------------------------------
# 复用Day17核心组件（无需修改，直接复用）
# ------------------------------------------------------------------------------
```

```python
# 1. Mask机制（因果掩码 + padding掩码）（Day9）
def create_padding_mask(seq, pad_idx=0):
    """padding掩码：屏蔽pad_idx对应的位置，shape [B, 1, 1, T]"""
    mask = (seq == pad_idx)
    return mask.unsqueeze(1).unsqueeze(2)

def create_causal_mask(seq_len):
    """因果掩码：屏蔽未来位置，确保生成时看不到后续token，shape [1, 1, T, T]"""
    mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool))
    mask = ~mask  # True=需要屏蔽，False=可访问
    return mask.unsqueeze(0).unsqueeze(1)

# 2. Scaled Dot-Product Attention（Day8）
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
        if mask is not None:
            scores = scores.masked_fill(mask, -1e9)
        attn_weights = F.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)
        return torch.matmul(attn_weights, v), attn_weights

# 3. Multi-Head Attention（Day10）
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0, "d_model必须能被n_heads整除"
        self.d_model = d_model
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads

        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        self.wo = nn.Linear(d_model, d_model)
        self.attn = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x):
        B, T, C = x.shape
        return x.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)

```

```python
    def forward(self, q, k, v, mask=None):
        B, T, C = q.shape
        q = self.split_heads(self.wq(q))
        k = self.split_heads(self.wk(k))
        v = self.split_heads(self.wv(v))
        attn_out, _ = self.attn(q, k, v, mask)
        attn_out = attn_out.transpose(1, 2).contiguous().view(B, T, C)
        return self.dropout(self.wo(attn_out))

# 4. Feed Forward (FFN) 网络 (Day12)
class FeedForward(nn.Module):
    def __init__(self, d_model, dropout=0.1, expansion=4):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_model * expansion)
        self.fc2 = nn.Linear(d_model * expansion, d_model)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        return self.fc2(self.dropout(self.gelu(self.fc1(x))))

# 5. Transformer Block (Pre-LN架构, Day15)
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, n_heads, dropout)
        self.ffn = FeedForward(d_model, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        residual = x
        x = self.norm1(x)
        x = residual + self.dropout(self.attn(x, x, x, mask))
        residual = x
        x = self.norm2(x)
        x = residual + self.dropout(self.ffn(x))
        return x

# 6. Embedding + 位置编码 (Day16)
class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size, d_model, pad_idx=0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_idx)
        self.scale = math.sqrt(d_model)

```

```python
    def forward(self, x):
        return self.embedding(x) * self.scale

class LearnablePE(nn.Module):
    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        self.pe = nn.Parameter(torch.randn(1, max_len, d_model))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        x = x + self.pe[:, :T]
        return self.dropout(x)

class SinCosPE(nn.Module):
    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x):
        B, T, C = x.shape
        x = x + self.pe[:T]
        return self.dropout(x)

# 7. 完整MiniGPT模型（Day17核心，直接复用）
class MiniGPT(nn.Module):
    def __init__(
        self,
        vocab_size,
        d_model=512,
        n_heads=8,
        n_layers=6,
        max_len=512,
        dropout=0.1,
        pad_idx=0,
        use_learnable_pe=True
    ):
        super().__init__()
        self.pad_idx = pad_idx
        self.max_len = max_len
```

```python
        self.d_model = d_model

        self.token_emb = TokenEmbedding(vocab_size, d_model, pad_idx)
        if use_learnable_pe:
            self.pos_emb = LearnablePE(d_model, max_len, dropout)
        else:
            self.pos_emb = SinCosPE(d_model, max_len, dropout)

        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_heads, dropout)
            for _ in range(n_layers)
        ])

        self.final_norm = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        self.token_emb.embedding.weight = self.lm_head.weight
        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, mean=0.0, std=0.02)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Embedding):
                nn.init.normal_(m.weight, mean=0.0, std=0.02)

    def forward(self, idx):
        B, T = idx.shape
        assert T <= self.max_len, f"序列长度{T}超过模型最大长度{self.max_len}"

        pad_mask = create_padding_mask(idx, self.pad_idx).to(idx.device)
        causal_mask = create_causal_mask(T).to(idx.device)
        mask = pad_mask | causal_mask

        x = self.token_emb(idx)
        x = self.pos_emb(x)

        for block in self.blocks:
            x = block(x, mask)

        x = self.final_norm(x)
        logits = self.lm_head(x)
        return logits

# ---------------------------------------------------------------------------
# 🔥 Day18 核心：前向推理测试（多场景、多设备、排错）
```

```python
# ------------------------------------------------------------------------------
def test_forward_pass(device, vocab_size=1000, d_model=128, n_heads=4, n_layers=2, max_len=64):
    """
    前向推理测试核心函数：适配多设备，验证模型输入输出合理性
    参数：device - 运行设备（cuda/mps/cpu），其余为模型超参数（简化版便于测试）
    返回：测试是否通过（bool）
    """
    print(f"\n=== 开始 {device} 设备前向推理测试 ===")
    try:
        # 1. 初始化模型（复用Day17模型，切换可学习/固定PE均测试）
        # 测试1：使用可学习位置编码（GPT标配）
        model_learnable = MiniGPT(
            vocab_size=vocab_size,
            d_model=d_model,
            n_heads=n_heads,
            n_layers=n_layers,
            max_len=max_len,
            use_learnable_pe=True
        ).to(device)

        # 测试2：使用sin/cos固定位置编码（验证组件兼容性）
        model_sincos = MiniGPT(
            vocab_size=vocab_size,
            d_model=d_model,
            n_heads=n_heads,
            n_layers=n_layers,
            max_len=max_len,
            use_learnable_pe=False
        ).to(device)

        # 2. 构造测试输入（3种场景，贴合周1Day6 GPT专用数据格式）
        # 场景1：基础输入（无padding，batch=2，seq_len=10）
        idx_base = torch.randint(1, vocab_size, (2, 10)).to(device)
        # 场景2：含padding输入（模拟真实batch，seq_len=15，含pad_idx=0）
        idx_pad = torch.randint(1, vocab_size, (2, 15)).to(device)
        idx_pad[:, 12:] = 0  # 最后3个位置设为padding
        # 场景3：极限长度输入（等于max_len，测试模型边界）
        idx_max = torch.randint(1, vocab_size, (1, max_len)).to(device)

        # 3. 前向推理（无梯度计算，快速验证）
        with torch.no_grad():
            # 测试可学习PE模型
            logits_learnable_base = model_learnable(idx_base)
            logits_learnable_pad = model_learnable(idx_pad)
            logits_learnable_max = model_learnable(idx_max)

```

```
            # 测试sin/cos PE模型
            logits_sincos_base = model_sincos(idx_base)

        # 4. 验证输出合理性（核心验收点）
        # 验证1：输出shape是否符合预期 [B, T, vocab_size]
        assert logits_learnable_base.shape == (2, 10, vocab_size), f"基础输入输出shape错误，预期(2,10,{vocab_size})，实际{logits_learnable_base.shape}"
        assert logits_learnable_pad.shape == (2, 15, vocab_size), f"含padding输入输出shape错误，预期(2,15,{vocab_size})，实际{logits_learnable_pad.shape}"
        assert logits_learnable_max.shape == (1, max_len, vocab_size), f"极限长度输入输出shape错误，预期(1,{max_len},{vocab_size})，实际{logits_learnable_max.shape}"
        assert logits_sincos_base.shape == (2, 10, vocab_size), f"sin/cos PE模型输出shape错误"

        # 验证2：输出值是否稳定（无NaN/Inf，避免梯度爆炸隐患）
        assert not torch.isnan(logits_learnable_base).any(), "可学习PE模型输出存在NaN"
        assert not torch.isinf(logits_learnable_base).any(), "可学习PE模型输出存在Inf"
        assert not torch.isnan(logits_sincos_base).any(), "sin/cos PE模型输出存在NaN"

        # 5. 测试通过提示
        print(f"✅ {device} 设备测试通过！")
        print(f"  - 可学习PE模型：3种输入场景均通过，输出shape正确、数值稳定")
        print(f"  - sin/cos PE模型：基础场景通过，组件兼容性验证成功")
        return True

    except Exception as e:
        # 6. 报错排查（Day18重点：常见问题及解决方案）
        print(f"❌ {device} 设备测试失败，错误信息：{str(e)}")
        if "shape" in str(e).lower():
            print("  排查建议：检查输入seq_len是否超过max_len，或d_model与n_heads是否整除")
        elif "device" in str(e).lower():
            print("  排查建议：确保输入tensor与模型在同一设备（使用.to(device)迁移）")
        elif "nan" in str(e).lower():
            print("  排查建议：检查dropout概率是否过高，或参数初始化异常")
        return False

if __name__ == "__main__":
    # 1. 设备列表（覆盖所有常见设备，贴合周1Day3 CUDA/MPS内容）
    devices = []
    if torch.cuda.is_available():
        devices.append(torch.device("cuda"))
```

```python
    if torch.backends.mps.is_available():
        devices.append(torch.device("mps"))
    devices.append(torch.device("cpu"))  # 兜底设备，确保所有环境可运行

    # 2. 统一超参数（简化版，适配测试速度，与Day17保持一致）
    vocab_size = 1000
    d_model = 128
    n_heads = 4
    n_layers = 2
    max_len = 64

    # 3. 执行多设备测试
    all_passed = True
    for device in devices:
        passed = test_forward_pass(device, vocab_size, d_model, n_heads, n_layers, max_len)
        if not passed:
            all_passed = False

    # 4. 整体验收总结
    print("\n" + "="*50)
    if all_passed:
        print("✅ Day18 模型前向推理测试全部通过！")
        print("✅ 已完成Day18目标：多设备验证模型可用性，排查常见报错，为后续任务铺垫")
        print("📌 后续可直接复用该模型，进行参数量统计（Day19）和生成功能开发（Day21）")
    else:
        print("❌ Day18 测试未全部通过，请根据上述错误提示排查问题后重新运行")
        print("📌 重点排查：设备迁移、输入shape、模型超参数兼容性")

```

四、Day18 关键要点（贴合计划，重点掌握）

1. 前向推理测试核心逻辑

测试流程：设备初始化 → 模型初始化（两种位置编码） → 构造多场景输入 → 前向推理 → 输出合理性验证（shape+数值稳定） → 报错排查

核心目的：确保Day17的模型可正常复用，无潜在bug，为后续训练、生成任务打基础。

2. 常见报错及排查方案（Day18重点，必掌握）

- 报错1：`RuntimeError: Expected all tensors to be on the same device`
  原因：模型和输入tensor不在同一设备（如模型在cuda，输入在cpu）

  解决方案：用`.to(device)`将输入和模型统一迁移到同一设备（周1Day3内容）

- 报错2：`AssertionError: 序列长度T超过模型最大长度max_len`
  原因：输入seq_len大于模型初始化时的max_len

  解决方案：要么缩短输入长度，要么重新初始化模型（增大max_len）

- 报错3：`RuntimeError: d_model必须能被n_heads整除`
  原因：多头注意力中，每个头的维度= d_model/n_heads，必须为整数

  解决方案：调整d_model或n_heads（如d_model=128、n_heads=4；d_model=512、n_heads=8）

- 报错4：`RuntimeError: Found NaN in input tensor`
  原因：参数初始化异常或dropout概率过高（如dropout=0.8）

  解决方案：降低dropout概率（建议0.1-0.3），检查参数初始化逻辑

3. 与计划的衔接说明

- 复用内容：Day17完整模型、周1Day3设备配置、周1Day6 GPT专用数据格式
- 为Day19铺垫：模型可正常运行，后续可直接统计参数量、分析模型复杂度
- 为Day21铺垫：前向推理稳定，后续只需在MiniGPT类中添加`generate`方法，即可实现文本生成
- 延伸周2内容：验证了Attention、FFN、Transformer Block等组件的兼容性，确保无串联bug

五、Day18 验收标准（必过）

1. 运行代码不报错，至少通过CPU设备测试（MPS/GPU设备可选，根据自身环境）
2. 3种输入场景（基础、含padding、极限长度）均能正常输出，shape符合预期`[B, T, vocab_size]`
3. 能独立排查前向推理中的常见报错（设备不一致、shape不匹配等）
4. 验证两种位置编码（可学习、sin/cos）的兼容性，模型均可正常推理

💡 Day18小结：今日核心是“测试验证”，通过多设备、多场景测试，确保Day17的模型可用、无bug。前向推理是大模型训练和生成的基础，今日过关后，后续Day19-21将逐步推进“参数量统计、模型保存加载、文本生成”，稳步实现“手写小GPT，能生成文本”的最终目标。

（注：文档部分内容可能由 AI 生成）